Relatório

INF1631 – Estruturas Discretas – Turma 3WB – 2026
Professor(a): Renata Lins
Alunos: Matheus Figueiredo - 2320813, Daniel Cipriani - 2310874, Ian Moura - 2310643

Parâmetros do grupo: **n = 10**, **p = 0,3** e **seed = 1376** (definidos no arquivo constantes.py).

1. Descrição da implementação

O programa foi feito em Python 3 e usa só a biblioteca padrão. Nenhuma biblioteca de grafos foi usada: todos os algoritmos principais foram implementados pelo grupo.

1.1 Representação do grafo

O grafo é guardado como uma lista de adjacência, ou seja, um dicionário {vértice: conjunto de vizinhos}, com os vértices de 1 a 10. Como o grafo não é direcionado, a aresta {u, v} aparece tanto em grafo[u] quanto em grafo[v]. Como os vizinhos ficam num conjunto, não tem como existir aresta paralela.

1.2 Funções obrigatórias (grafos.py)

gerar_grafo(numero_de_vertices, probabilidade, random_com_seed): percorre os C(10,2) = 45 pares de vértices com u < v e inclui a aresta quando random_com_seed.random() < p (função auxiliar sorteia_aresta). Como só pares distintos são sorteados e cada par uma vez só, o grafo gerado é sempre simples (sem laços e sem arestas paralelas).
graus(grafo): devolve um dicionário vértice -> grau, onde o grau é a quantidade de vizinhos do vértice.
conexo(grafo): faz uma busca em largura (BFS) começando de um vértice de grau maior que zero e verifica se ela chega em todos os vértices de grau maior que zero. Vértices isolados são ignorados, como pede o enunciado.
euleriano(grafo): usa explicitamente os dois critérios matemáticos: conexo(grafo) e todos_graus_pares(grafo). Também trata o caso do grafo vazio.
circuito_euler(grafo): constrói o circuito com o algoritmo de Hierholzer usando uma pilha, trabalhando numa cópia do grafo para não estragar o original. Devolve a sequência de vértices do circuito, ou None se o grafo não for euleriano.
valida_circuito(grafo, circuito): confere de forma independente se o circuito começa e termina no mesmo vértice, se cada passo é uma aresta do grafo, se nenhuma aresta se repete e se todas as arestas foram usadas.
Além dessas, existem algumas funções auxiliares: cria_grafo (grafo com n vértices e sem arestas), sorteia_aresta, arestas(grafo) (lista ordenada das arestas) e todos_graus_pares(grafo). O arquivo resultados.py guarda os resultados dos experimentos, tempo.py tem a classe Cronometro usada para medir o tempo do experimento 2 e util.py concentra as funções de impressão da saída.

1.3 Experimentos (main.py)

Experimento 1: cria um random.Random(1376) e vai gerando grafos com p = 0,3 até que euleriano(grafo) seja verdadeiro. Depois imprime as arestas, os graus, os critérios, o circuito encontrado pelo Hierholzer e o resultado da validação automática.
Experimento 2: cria um novo random.Random(1376), para que esse experimento também possa ser reproduzido sozinho, e gera 1000 grafos. Para cada um verifica se é euleriano. No final mostra o tempo gasto, quantos grafos eram eulerianos e a proporção. Também foram contados separadamente os grafos com todos os graus pares e os grafos conexos, para ajudar na análise.

1.4 Decisões do grupo

Grafo vazio: consideramos um grafo sem arestas como rede inválida, já que nesse caso o robô não teria nenhuma rua para inspecionar.
Seed: cada experimento cria seu próprio gerador com a seed 1376. Assim, rodar o programa duas vezes gera exatamente a mesma saída (menos os tempos, que dependem da máquina).

2. Testes obrigatórios

Os testes estão no arquivo testes.py e são executados com o comando python3 testes.py. Todos os 11 testes passaram (4 obrigatórios e 7 complementares).
Grafo com Euler (ciclo C4: 1-2-3-4-1): o grafo é euleriano e o circuito encontrado foi 1 -> 2 -> 3 -> 4 -> 1, considerado válido.
Vértices ímpares (caminho P4: 1-2-3-4): o grafo é conexo, mas os vértices 1 e 4 têm grau 1, então não possui circuito de Euler.
Grafo desconexo (dois triângulos separados, 1-2-3 e 4-5-6): todos os graus são pares, mas o grafo é desconexo, então não possui circuito de Euler.
Grafo vazio (10 vértices e nenhuma aresta): tratado explicitamente como rede inválida.
Os testes complementares verificam os casos extremos p = 0 e p = 1, um grafo onde o Hierholzer precisa juntar subciclos (dois triângulos com um vértice em comum), um grafo com vértices isolados, o K5 e se valida_circuito rejeita circuitos que não fecham, que usam arestas que não existem ou que repetem arestas.

3. Resultados

3.1 Experimento 1 – primeiro grafo euleriano

Número de vértices: 10
Probabilidade p: 0.30
Seed: 1376

Grafo Euleriano encontrado!
Número de grafos gerados até encontrá-lo: **1347**

Arestas (14):
(1,2) (1,4) (2,8) (3,7) (3,8) (3,9) (3,10) (4,6) (5,7) (5,8) (6,8) (7,8) (7,9) (8,10)

Graus:
v1: 2, v2: 2, v3: 4, v4: 2, v5: 2, v6: 2, v7: 4, v8: 6, v9: 2, v10: 2

Conexo: sim
Todos os graus são pares: sim
Euleriano: sim

Circuito de Euler:
1 -> 2 -> 8 -> 3 -> 7 -> 5 -> 8 -> 7 -> 9 -> 3 -> 10 -> 8 -> 6 -> 4 -> 1

Validação do circuito: **OK**

3.2 Experimento 2 – 1000 grafos

Tempo de geração dos grafos: 0.0069 s
Tempo total (geração + verificação): 0.0321 s

Grafos com todos os graus pares: 0

Grafos conexos (vértices de grau não nulo): 935
Grafos vazios (rede inválida): 0

Grafos Eulerianos: **0 de 1000**
Proporção: **0.000**
Os tempos mudam um pouco a cada execução e dependem do computador; os outros números são sempre os mesmos.

4. Análise dos resultados

4.1 O grafo euleriano encontrado

O primeiro grafo euleriano encontrado tem 14 arestas, o que é próximo do esperado ($p \cdot 45 = 13{,}5$). Todos os 10 vértices têm grau maior que zero, ou seja, não tem vértice isolado, e todos os graus são pares: sete vértices de grau 2, dois de grau 4 (v3 e v7) e um de grau 6 (v8). A soma dos graus dá $28 = 2 \cdot 14$, como diz o lema do aperto de mãos.
O vértice 8 funciona como uma espécie de "praça central": o circuito passa por ele três vezes (grau 6 / 2), e pelos vértices 3 e 7 duas vezes cada. O circuito tem 15 posições (com repetições de vértices) e percorre as 14 arestas. Ele começa e termina no vértice 1, e a validação automática confirmou que cada aresta foi usada exatamente uma vez.

4.2 Quantidade de grafos gerados

Foram necessários 1347 grafos até achar o primeiro euleriano.

4.3 Proporção nos 1000 experimentos

Nenhum dos 1000 grafos foi euleriano (proporção 0,000). Esse resultado faz sentido por dois motivos.
Primeiro, o experimento 2 usa a mesma seed do experimento 1, então ele gera exatamente os mesmos 1000 primeiros grafos. Como o primeiro euleriano só apareceu no grafo 1347, nenhum dos 1000 primeiros poderia ser euleriano.
Segundo, mesmo sem isso, a proporção esperada já é muito baixa. Para conferir, fora do experimento obrigatório, a mesma função experimento_proporcao foi rodada com mais grafos: com 20.000 grafos e seed 1376 saíram 39 eulerianos (cerca de 0,00195), 39 com todos os graus pares e 18.804 conexos; com 200.000 grafos e seed 2026 saíram 393 eulerianos (cerca de 0,00197), 400 com todos os graus pares e 187.967 conexos.
Ou seja, o esperado é algo em torno de 2 grafos eulerianos a cada 1000. Usando essa probabilidade estimada ($q \approx 0{,}00197$), a chance de nenhum dos 1000 grafos ser euleriano é mais ou menos $(1 - q)^{1000} \approx 0{,}14$, ou seja, uns 14%. Então ter dado 0 numa amostra de 1000 é bem plausível.

4.4 Circuito de Euler e graus dos vértices

A condição de todos os graus serem pares foi o principal obstáculo. Nos 1000 grafos, nenhum tinha todos os graus pares; em média cada grafo tinha uns 5 vértices de grau ímpar. Basta um único vértice de grau ímpar para não existir circuito: o robô entraria nesse cruzamento uma vez a mais do que conseguiria sair, e ficaria preso lá ou teria que repetir uma rua.
Como o grau de cada vértice é a soma de 9 sorteios independentes, cada grau tem mais ou menos 50% de chance de ser par. Exigir que os 10 sejam pares ao mesmo tempo é muito restritivo, e a probabilidade fica perto de $(1/2)^9 \approx 0{,}002$, que bate com o que foi observado. O expoente é 9 e não 10 porque a soma dos graus é sempre par, então a paridade do último vértice já fica determinada pelos outros nove.

4.5 Circuito de Euler e conectividade

A conectividade também é necessária, mas com esses parâmetros ela é um filtro bem mais fraco: 935 dos 1000 grafos (93,5%) tinham todos os vértices de grau não nulo na mesma componente. O teste dos dois triângulos mostra que só ter graus pares não basta: se a rede tiver duas regiões sem ligação entre elas, o robô nunca consegue chegar nas ruas da outra região, mesmo que todos os cruzamentos tenham grau par.
Na simulação com 200.000 grafos, a quantidade de grafos com todos os graus pares (400) e a de grafos eulerianos (393) ficaram quase iguais: só 7 grafos com graus pares não eram conexos. Ou seja, com esses parâmetros quase todo grafo com graus pares já é conexo. Então a paridade foi o critério mais restritivo, mas a conectividade continua sendo necessária.

4.6 Conclusão

O programa gera grafos aleatórios, decide se existe circuito de Euler usando os dois critérios matemáticos, constrói o circuito com o algoritmo de Hierholzer e valida o circuito automaticamente. Com n = 10 e p = 0,3, uma rede sorteada ao acaso quase nunca permite que o robô passe por todas as ruas uma única vez e volte para o início, e o que mais pesa nisso é a exigência de que todos os cruzamentos tenham grau par.

5. Como executar

As instruções para executar o programa estão no arquivo README.md. Resumindo: python3 main.py roda os dois experimentos e python3 testes.py roda os testes.